GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-29

今日榜单

1. **HKUDS/DeepTutor** — Python
   DeepTutor: Lifelong Personalized Tutoring.https://deeptutor.info/. — 3天
2. **obra/superpowers** — Shell
   An agentic skills framework & software development methodology that works. — 3天
3. **pbakaus/impeccable** — Rust
   The design vocabulary for your coding agent. Steer with polish and bolder, revie... — 6天
4. **JetBrains/thinkrail** — TypeScript
   Vibe code with pi in a lightweight, real IDE that customises itself around the w... — 5天
5. **milind-soni/OpenMausBot** — TypeScript
   Open-source Grok Bot alternative with a virtual machine that bots can use — NEW
6. **nightly-labs/openbot** — TypeScript
   A local-first desktop workspace for persistent AI teammates. Run Codex, Claude, ... — 4天
7. **elayadesign/ai-design-skills** — Unknown — 2天
8. **thedotmack/claude-mem** — TypeScript
   Persistent Context Across Sessions for Every Agent – Captures everything your ag... — NEW
9. **CopilotKit/OpenBot** — TypeScript
   Open-source AI coworkers that each get a computer of their own: a browser, files... — NEW
10. **togg53192-cmd/jailbreaks** — Unknown
    LIST OF ALL MY JAILBREAKS — 3天

#1 Python 连续 3 天

DeepTutor

DeepTutor: Lifelong Personalized Tutoring.https://deeptutor.info/.

Stars **49,503** Forks **6,094** Today **+1,926**

https://github.com/HKUDS/DeepTutor

项目简介：DeepTutor 是香港大学数据科学实验室（HKUDS）推出的"终身个性化辅导"开源项目，目标是把 AI 从一次性问答工具变成能长期记住你、伴随你成长的学习伙伴。它试图解决当前 AI 教育产品"每次对话都从零开始、缺乏学习连续性"的核心痛点。

核心功能：围绕"终身个性化"展开，主要包括：持续积累的学习者画像与记忆机制（跨会话保留学习进度与偏好）、个性化学习路径与辅导策略生成、多语言界面支持（README 提供 12 种语言版本，明显面向全球用户）、以及 Agent 原生的 CLI 接口，便于开发者集成与二次开发。项目还规划了 EduHub 技能社区生态，让辅导能力可以像插件一样扩展。

适用场景：适合需要长期陪伴式学习的个人用户（如备考、语言学习、技能进阶），也适合教育机构、AI 应用开发者作为个性化辅导系统的底座来二次开发。对研究者而言，它也是研究"长期记忆 + 个性化 Agent"的实践样本。

技术亮点：采用 Python 3.11+ 后端搭配 Next.js 16 前端的现代技术栈，Apache 2.0 开源协议，并附有 arXiv 论文（2604.26962），说明其背后有学术方法支撑而非纯工程拼装。CLI 被定位为"Agent 原生接口"，暗示其架构以 Agent 编排为核心，便于被其他 AI 系统调用。近 5 万星标、单日新增近 2000，热度极高，但需注意这类爆款项目的成熟度往往滞后于关注度。

上手建议：先访问官网 deeptutor.info 阅读文档，按 Get Started 跑通本地部署，再通过 CLI 体验核心辅导流程；想深入可对照 arXiv 论文理解设计思路。贡献方面，建议从 Roadmap issue 投票或认领小任务入手，并先读 CONTRIBUTING.md 了解分支策略与代码规范；社区有 Discord、飞书、微信群，适合先潜水观察项目节奏再动手。

#2 Shell 连续 3 天

superpowers

An agentic skills framework & software development methodology that works.

Stars **308,168** Forks **27,414** Today **+1,037**

https://github.com/obra/superpowers

项目简介：Superpowers 是一套面向 AI 编程智能体的“软件开发方法论”框架，由一系列可组合的 skill（技能）和初始指令构成，核心目的是让编程智能体不再一上来就写代码，而是先澄清需求、产出规格与实现计划，再按计划执行。它解决的是 AI 编码助手“急于动手、缺乏工程纪律”的普遍问题。

核心功能：一是需求澄清与规格输出，智能体先反问用户真实意图，再分块展示设计供人确认；二是生成“足够傻瓜化”的实现计划，强调红绿 TDD、YAGNI、DRY；三是子智能体驱动的开发流程（subagent-driven-development），由多个智能体分头执行任务并互相审查，可长时间自主推进；四是技能自动触发，无需用户额外操作；五是提供跨多种 harness（Claude Code、Cursor、Gemini CLI、Codex、Copilot CLI 等）的安装方式。

适用场景：适合重度使用 AI 编程智能体做中大型功能开发的工程师与团队，尤其是希望 AI 在无人值守下按计划稳定推进数小时、且产出可审查代码的场景；也适合想把“规格先行、TDD、小步评审”这套工程纪律固化进 AI 工作流的团队。

技术亮点：项目本身用 Shell 编写，本质是“提示词工程 + 技能编排”而非传统代码库，通过可组合 skill 和初始指令实现方法论注入；其子智能体分工与评审机制把多智能体协作落到了具体开发流程上；同时适配十余种主流 CLI/IDE 智能体，具备较强的 harness 无关性。

上手建议：先根据你使用的工具（如 Claude Code 用 `/plugin install superpowers@claude-plugins-official`）完成安装，然后从一个真实的小需求走一遍“澄清→规格→计划→执行”的完整流程，观察智能体行为变化；想贡献可从 README 的 Contributing 与 Philosophy 章节入手，理解其技能设计理念后再新增或改进 skill。

#3 Rust 连续 6 天

impeccable

The design vocabulary for your coding agent. Steer with polish and bolder, review with critique and audit, catch AI slop before it ships.

Stars **94,746** Forks **5,593** Today **+2,881**

https://github.com/pbakaus/impeccable

项目简介：Impeccable 是一套为 AI 编程助手设计的“设计词汇表”，通过一组标准化命令（如 polish、bolder、critique）来引导 Claude Code、Cursor 等工具产出更有品味的界面。它解决的核心问题是：AI 生成的前端页面往往陷入“奶油底色 + 细衬线 + 渐变按钮”的千篇一律审美，缺乏真正的设计判断力。

核心功能： - **引导类命令**：`bolder/quieter/distill/colorize` 等，用精确动词替代“让它更亮眼”这类模糊指令，为每种操作加载专门的“操作手册”并指向具体目标。 - **审查类命令**：`critique` 按可用性启发式打分并排序待修问题，`audit` 检查无障碍、性能、主题和响应式，发现项按 P0–P3 分级。 - **AI 套路检测器**：62 条确定性规则识别“AI 味”设计（嵌套卡片、彩色背景上的灰字、圆角图标块等），纯规则实现，不依赖 LLM 或 API key。 - **上下文文件机制**：`init` 生成 `PRODUCT.md` 记录产品定位，`document` 生成 `DESIGN.md` 固化现有视觉系统，后续命令都读取这两份文件保持一致性。 - **实时迭代**：`live` 支持在浏览器中点选元素循环切换变体，`generate` 无需手动挑选即可生成变体。

适用场景：主要面向使用 AI 编程助手做前端开发的产品设计师、独立开发者和前端工程师，尤其适合“已有界面需要打磨”的迭代场景——指向某个组件或页面反复优化。新建页面也能用，但官方坦言这是更难的模式，效果不如在既有界面上迭代。

技术亮点：值得关注的是它的“反 LLM”路线——slop 检测器用 62 条确定性规则而非模型判断，保证了可复现性和零成本；同时项目用 Rust 编写（尽管 README 主体是命令层），说明底层工具链追求性能与分发便利。另外它把“设计品味”拆解成 24 个可组合动词，本质是把模糊的设计意图工程化为可执行指令，这个思路比单纯堆 prompt 更有结构性。

上手建议：从项目根目录执行 `npx impeccable install`，重载工具后跑一次 `/impeccable init` 建立产品上下文，然后挑一个具体组件试 `critique`，感受评分与修复排序是否符合预期。想深入的话，先读官方文档里的 context files 和 24 个命令各自的页面；贡献方向可关注 slop 规则库的扩充和命令 playbook 的质量——这两处直接决定工具的实际效果。

#4 TypeScript 连续 5 天

thinkrail

Vibe code with pi in a lightweight, real IDE that customises itself around the way you work — The Vibe You Need

Stars **5,811** Forks **306** Today **+1,116**

https://github.com/JetBrains/thinkrail

项目简介：ThinkRail 是 JetBrains 孵化的一款"智能体 IDE"（agentic IDE），围绕 pi 编码智能体构建了完整的图形界面。它要解决的核心问题是：现有 AI 编码工具多是"聊天框 + 编辑器"的松耦合拼装，缺乏对项目意图的持久记忆和隔离的并行工作流。ThinkRail 试图让 IDE 随使用不断"学习"你的项目，从"活的规格文档"到未来自建扩展能力。

核心功能：一是基于 `git worktree` 的隔离工作区，每个智能体会话在自己的分支和目录中并行工作，互不干扰，完成后可一键合并或丢弃；二是围绕智能体的真实 IDE 工作台，包含 Monaco 编辑器、文件树、终端、变更视图，并支持在 diff 上留锚定评论后作为结构化上下文发给对话；三是多会话并发的 pi 聊天，各自独立选模型、实时显示 token 与成本，支持中途干预、排队追问、委派子智能体，并共享一份可编辑的执行计划；四是以 SPEC.md 文件图作为"事实来源"，智能体通过专用 `spec_*` 工具读写维护，让意图跨会话留存；五是兼容已有的 Agent Skills 与内置工作流（项目搭建、设计头脑风暴、发布 PR）。

适用场景：主要面向希望把 AI 智能体深度嵌入日常开发流程的工程师和团队，尤其是需要并行推进多个分支任务、又不想污染主工作区的场景。也适合重视"项目知识沉淀"的团队——用规格文档图谱让 AI 理解长期意图，而非每次从零开始。对 JetBrains 生态用户和想尝鲜"下一代 IDE 形态"的开发者有较强吸引力。

技术亮点：最值得关注的是"隔离 + 持久知识"的双层设计——用 git worktree 做物理隔离保证并行安全，用 SPEC.md 图谱做语义隔离保证上下文不丢失，这两点切中了当前 AI 编码工具的真实痛点。此外，把 PR 生命周期（CI 检查、评审决策、合并、失败处理）直接内嵌到工作台并交给智能体接管，是把"编码"和"交付"打通的有意思尝试。项目以 TypeScript 实现，复用 pi 作为智能体内核而非自研，属于务实的工程取舍。

上手建议：从官网 thinkrail.ai 和 README 的截图入手理解工作台布局，先在一个小仓库上体验"建工作区 → 跑 pi 会话 → 看 diff → 发 PR"的完整闭环。想深入可读其博客了解设计思路，并加入 Discord

社区。贡献方面，项目仍处孵化期且明确规划了"扩展支持"，关注 issue 中关于扩展机制和 `spec_*` 工具的方向，是参与的好切入点。注意它目前更像早期产品，适合尝鲜而非生产强依赖。

#5 TypeScript NEW

OpenMausBot

Open-source Grok Bot alternative with a virtual machine that bots can use

Stars **7,244** Forks **1,092** Today **+1,190**

https://github.com/milind-soni/OpenMausBot

项目简介：OpenMausBot 是一个开源的"Grok Bot 替代品"，把 AI 智能体包装成聊天应用里的"联系人"——侧边栏每个 bot 都是真实运行的 Claude 或 Codex 智能体，各有独立人格、模型、云端电脑和已连接的应用。它解决的是"单一助手塞进一个对话框"这种形态不匹配问题，让用户像发消息一样与多个专职智能体协作，且本地优先、自带模型（BYO agent）。

核心功能：① 多智能体聊天列表，每个 bot 拥有独立人格、记忆线程、模型与云电脑；② 内置虚拟机，bot 可在其中执行真实任务，用户可实时观看其工作过程；③ 关键操作需人工审批（approve what matters），兼顾自主性与可控性；④ 可连接第三方应用，扩展智能体能力边界；⑤ 跨平台桌面客户端（macOS/Windows/Ubuntu），带签名公证的安装包。

适用场景：适合希望摆脱单一聊天机器人、想要"一支 AI 团队"的个人开发者和重度 AI 用户；典型场景包括让不同 bot 分管编码、调研、运维等任务，在本地或自带模型上运行以保护隐私，以及需要可视化监督智能体执行过程的研究与自动化工作流。

技术亮点：采用 TypeScript 严格模式 + React 19 + Electron 构建跨平台桌面应用；架构上把 Claude/Codex 作为底层执行引擎在本地运行，bot 只是其上的"人格+配置"封装，实现 bring-your-own-agent 的本地优先设计；虚拟机隔离为智能体提供安全的执行沙箱，审批机制则在自主与安全间取得平衡。

上手建议：普通用户直接从官网或 Releases 下载对应平台安装包即可体验；开发者建议先阅读 README 中"Bring your own agents"部分，确认本地已配置 `claude`/`codex` 环境，再从项目结构入手理解 bot 配置与虚拟机集成方式，PR 欢迎，可先从小 issue 或文档改进切入。

#6 TypeScript 连续 4 天

openbot

A local-first desktop workspace for persistent AI teammates. Run Codex, Claude, and Grok with dedicated workspaces, task queues, file sharing, browser control, and agent-to-agent collaboration.

Stars **3,706** Forks **347** Today **+705**

https://github.com/nightly-labs/openbot

项目简介：OpenBot 是一个本地优先的桌面工作区，把 Codex、Claude、Grok 等多个 AI 编程代理（agent）组织成"持久化的 AI 队友"，每个代理拥有独立的工作区和会话。它解决的是当前 AI 编码代理各自为战、缺乏统一调度和协作基础设施的问题，让多个代理能像团队成员一样分工、通信和共享文件。

核心功能：一是多代理持久化会话，每个 agent 绑定独立的 Codex/Claude/Grok 等后端和本地工作区，并带上下文监控与自动压缩；二是 FIFO 任务队列，支持暂停、恢复、取消和崩溃安全持久化；三是代理间消息、回复、文件传输和共享频道协作，含任务负责人、委派和重新分配控制；四是内置持久化浏览器，代理可打开、检查并控制网页；五是可选的 Computer Use 能力（通过 cua-driver）让代理操作桌面。

适用场景：适合需要同时驱动多个 AI 编码代理完成复杂工程任务的开发者，比如让一个代理写代码、另一个做代码审查、再一个跑测试并互相传文件的流水线式协作。也适合想用本地模型（Ollama、LM Studio）或自建 OpenAI 兼容端点、注重数据留在本地的团队。由于目前代理以完全权限运行，更适合在可信任务和可备份的环境中试用。

技术亮点：用 TypeScript 构建跨平台桌面应用，通过 ACP 协议统一接入 Grok、OpenCode、Gemini、Cursor、Cline 等多种代理后端，抽象层设计得比较通用；本地优先架构配合 Cloudflare Workers + D1 的可选账号服务，兼顾隐私与轻量云端能力；队列的崩溃安全持久化和代理间消息机制，是把它从"多开几个聊天窗口"提升为"协作工作区"的关键工程点。

上手建议：先从 GitHub Releases 下载对应平台安装包（macOS/Windows/Linux AppImage），注意 Linux 上 Ubuntu 23.10+ 和 Debian 13 需要安装随附的 AppArmor profile；首次启动会要求明确同意完全权限模式，务必先读 README 的 Security 部分并做好备份。想贡献代码可从 CI 配置和 ACP 适配层入手，理解代理接入的抽象接口是参与开发的最佳切入点。

#7 Unknown 连续 2 天

ai-design-skills

Stars **11,889** Forks **861** Today **+1,178**

https://github.com/elayadesign/ai-design-skills

项目简介：这是一个面向 AI 编程工具的设计技能（Design Skills）集合，本质是一批精心编写的 Markdown 规则文件，让 Claude Code、Cursor、Codex 等工具在写代码时能遵循专业的设计规范。它解决的核心问题是：AI 生成的页面往往"能跑但难看"，缺乏排版、间距、动效等设计系统层面的约束。

核心功能：目前主打 `landing-page-design` 技能，可从零构建落地页，涵盖需求引导问题、页面结构、转化文案，以及完整的视觉系统（字体、间距、圆角、动效）。每个技能是自包含的文件夹，按需安装。设计价值观集中在文件底部一个区块，改这一处即可整体定制风格。另有独立的 `redesign-skill` 仓库，专门处理改造现有站点而非新建。

适用场景：适合用 AI 编程工具做产品落地页、营销页的独立开发者、设计师和小团队；也适合希望给 AI 输出"上设计约束"、避免千篇一律模板感的工程师。不写代码、只在 Claude.ai 网页端用的人同样能通过上传文件使用。

技术亮点：项目本身几乎不含代码，价值全在"提示词工程 + 设计系统"的封装上——把设计决策（字体、色彩、间距、动效）结构化成 AI 可读的规则，并通过"单一配置区块"实现一处修改、全局生效的可定制性。多工具适配做得务实，同一份 `SKILL.md` 可映射到 `.claude/skills`、`.cursor/rules`、`AGENTS.md`、`.windsurfrules` 等不同路径，降低了跨工具迁移成本。

上手建议：从 `landing-page-design` 入手，按 README 对应你所用工具的一条安装命令操作，然后用自然语言直接描述需求即可。想定制风格，就 fork 仓库后修改技能文件底部的设计价值观区块；想贡献，可参照现有技能的自包含文件夹结构新增技能，并保持"设计决策集中在一处"的组织方式。

#8 TypeScript NEW

claude-mem

Persistent Context Across Sessions for Every Agent – Captures everything your agent does during sessions, compresses it with AI, and injects relevant context back into future sessions. Works with Claude Code, OpenClaw, Codex, Gemini, Hermes, Copilot, OpenCode + More

Stars **104,422** Forks **9,217** Today **+459**

https://github.com/thedotmack/claude-mem

项目简介：claude-mem 是一个为 AI 编程智能体（尤其是 Claude Code）设计的持久化记忆系统。它解决了智能体在会话结束后"失忆"的问题——每次新会话都要重新交代背景、重复解释项目结构。项目通过捕获会话过程、用 AI 压缩成可检索的记忆，再在后续会话中自动注入相关上下文，让智能体真正"记住"之前做过什么。

核心功能：第一，会话全量捕获——记录智能体在一次会话中的所有操作、决策和代码变更；第二，AI 压缩——把冗长的原始记录提炼成精简、结构化的记忆条目，避免上下文窗口被撑爆；第三，相关性注入——新会话开始时按需检索并注入历史上下文，而非无脑全塞；第四，多智能体兼容——除 Claude Code 外还支持 Codex、Gemini、Copilot、OpenCode 等主流工具。

适用场景：主要面向重度使用 AI 编程助手的开发者，尤其是跨天、跨周维护同一项目的场景——比如第二天继续让智能体接着昨天的重构任务干活，或让新开的会话立刻理解项目约定和踩过的坑。团队协作中也能让不同成员、不同智能体共享同一份项目记忆。

技术亮点：用 TypeScript 实现，核心思路是"捕获—压缩—检索"三段式流水线，把原始会话日志转化为可复用的记忆资产，而不是简单拼接历史对话。压缩环节借助 AI 完成，这是控制 token 成本的关键设计；同时以插件/适配层方式对接多个智能体，说明其抽象做得比较克制、可扩展。项目版本已迭代到 15.x，Apache 2.0 协议，工程成熟度不低。

上手建议：从 README 的 Quick Start 入手，先以 Claude Code 为单一目标跑通"安装—开启一次会话—验证记忆注入"的闭环，确认效果后再扩展到其他智能体。想贡献代码的话，建议先读 `docs/` 下的架构说明和多语言文档，理解捕获与压缩的数据流，再从适配新智能体或优化压缩策略这类边界清晰的模块切入。

#9 TypeScript NEW

OpenBot

Open-source AI coworkers that each get a computer of their own: a browser, files and tools, with every action decided before it happens and recorded after. Bring any AG-UI agent.

Stars **7,917** Forks **1,038** Today **+240**

https://github.com/CopilotKit/OpenBot

项目简介

OpenBot 是 CopilotKit 推出的开源 AI 同事平台，定位为"企业真正能拥有的 AI 助手"——形态类似 ChatGPT/Claude，但完全跑在自有基础设施上，代码可任意修改。它解决的核心问题是：企业想要 AI 助手的能力，却不愿把数据和工具控制权交给第三方 SaaS，同时希望每个 AI 助手有独立、可审计的操作环境。

核心功能

- **每个 AI 同事独占一台"电脑"**：拥有独立的真实浏览器（含独立登录态）、文件系统和被授权的工具集，可观察其屏幕操作。
- **人在环中接管**：AI 执行到敏感动作前先决策、事后全程记录，用户可随时"接手方向盘"再交还，形成可审计的行为链。
- **AG-UI 协议接入任意 Agent**：任何符合 AG-UI 协议的 agent（LangGraph 等框架构建或手写）都能作为"同事"接入，并拥有独立频道。
- **组件化回答**：AI 不只输出文字，还能以 UI 组件形式回应。
- **开箱即用的多角色配置**：默认带 General Assistant、Knowledge 两个同事，`examples/fintech/` 下另有 10+ 个垂直角色（费用审核、会议纪要、发布说明、工单分流等），以配置而非代码形式扩展。

适用场景

- **对数据主权敏感的企业**：金融、医疗、法务等需要 AI 在自有 PostgreSQL 和基础设施内运行、模型凭证自管的团队。

- **需要可审计 AI 操作的场景**：让 AI 接触真实工具（浏览器、文件、MCP 服务）但必须"先决策后记录"的合规型业务。
- **想自建 AI 产品形态的团队**：把 OpenBot 当模板克隆，替换 `examples/` 下的租户包，定制成自家产品。

技术亮点

- **统一网关治理**：所有对电脑、文件、MCP 服务器和组件的操作都经过同一网关做决策与记录，这是"能用工具"与"敢让它碰工具"的分界线。
- **协议解耦**：基于 AG-UI 开放协议，不绑定任何 agent 框架，前后端与 agent 实现可独立演进。
- **Docker Compose 一键起全栈**：数据落自有 PostgreSQL，模型不预置、凭证加密存储且不写日志；默认 `OPENBOT_SINGLE_USER=true` 让新克隆免 OAuth 直接可用。
- **模板而非产品**：仓库内所有 workspace 均为 private，不发布 npm 包，强制"fork 后改造"的使用范式，避免被当成托管服务依赖。

上手建议

从 README 的 Quick Start 开始：克隆仓库、复制 `.env.example`、用 Docker Compose 拉起全栈，在 Mac 上可用 CopilotKit 免费 Developer plan 本地跑 Intelligence。接着读 `docs/README.md`，把 `examples/fintech/agents/` 里的示例角色复制进来试跑，再替换成自己的同事、频道和技能。项目处于 Beta 且活跃开发，适合先以"克隆改造"心态试用，遇到问题提 Issue/PR；若要在企业内部落地，官方也提供工程师协助部署与定制的通道。

#10 Unknown 连续 3 天

jailbreaks

LIST OF ALL MY JAILBREAKS

Stars **8,364** Forks **1,173** Today **+1,610**

https://github.com/togg53192-cmd/jailbreaks

项目简介：这是一个汇集了32套系统提示词（system prompt）的仓库，覆盖Claude、Gemini、Grok、DeepSeek、GLM、Kimi、Qwen等26+个前沿及开源模型。它解决的核心问题是：为研究者提供"完整可粘贴"的提示词样本，用于观察指令层级、人格设定与策略覆盖如何影响模型行为。

核心功能：一是按模型家族分类的完整系统提示词集合（含人格锁定、工作区策略、AGENTS.md文件、Ollama Modelfiles）；二是针对OpenCode Zen免费"隐身模型"的专用配置；三是提供编码代理与工作区指令格式的参考模板；四是以"paste-ready"为原则保证每个文件完整可用、无省略。

适用场景：主要面向AI安全研究者、提示词工程师和红队测试人员，用于研究提示注入防御、指令冲突与策略覆盖机制；开发者也可借鉴其AGENTS.md与配置块格式来设计自己的代理工作流。普通用户若想"越狱"使用则需注意仓库本身声明仅供研究教育用途。

技术亮点：值得关注的是它把"越狱"重新框定为指令层级与人格框架（persona framing）的研究素材，而非单纯的绕过技巧——这对理解模型如何解析冲突指令、如何被策略覆盖有直接价值。此外，它同时覆盖闭源前沿模型与开源权重模型，并包含Ollama Modelfile等落地格式，样本维度较全。

上手建议：先读README的Collection表格定位目标模型，从对应.md文件入手理解提示词结构；若关注防御侧，可重点对比同一模型不同版本（如opus v1/v2）的差异。贡献方面，仓库标注PRs Welcome，可补充新模型样本或修正失效提示词，但需遵守其"不用于伤害他人或违反服务条款"的声明。

数据来源：github.com/trending

报告日期：2026-10-29

由 DeepSeek AI 提供智能分析 | 自动生成